

AI SCIENTIST

FASE 2 | AULA 03 - OTIMIZAÇÃO DE QUERIES: ÍNDICES, PARTIÇÕES E BOAS PRÁTICAS PARA CONSULTAS RÁPIDAS

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

MERCADO, CASES E TENDÊNCIAS 12

O QUE VOCÊ VIU NESTA AULA? 14

REFERÊNCIAS 15

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Em escala, a diferença entre uma query bem escrita e uma mal planejada é a diferença entre um insight em 5 segundos ou um sistema travado por 5 horas. Nós abordaremos a fronteira final da proficiência em SQL para Ciência de Dados: a performance.

Nesta aula, o foco é a velocidade. Não basta o resultado estar correto, ele precisa ser entregue em tempo hábil para a tomada de decisão e que não prejudique no funcionamento base de outras aplicações.

Vamos explorar como o MySQL organiza os dados fisicamente e como podemos usar Índices e Particionamento para evitar o pesadelo de qualquer DBA: o Full Table Scan. Nós vamos ver novamente um comando que falamos brevemente, mas agora você aprenderá a ler o "pensamento" do banco através do comando EXPLAIN e a aplicar boas práticas que reduzem drasticamente o consumo de recursos.

Mas o que acontece quando sua consulta precisa rodar em uma tabela de 500 milhões de registros? O que diferencia um(a) profissional comum de um(a) especialista é a capacidade de entender como o banco de dados armazena as informações fisicamente e como "ajudar" o otimizador a encontrar o caminho mais rápido. Vamos aprender que **velocidade não é sorte, é design.**

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Utilizando o nosso banco **GlobalTech E-commerce** (agora com um volume de dados simulado muito maior), vocês irão atuar em três frentes:

- **O diagnóstico:** receberemos uma query propositalmente lenta (com múltiplos JOINs e filtros não otimizados). O desafio será usar o comando EXPLAIN para descobrir onde está o gargalo (Full Table Scan vs. Index Seek).
- **A intervenção:** criaremos índices estratégicos, incluindo **Índices Compostos**, e observaremos a mudança no "Custo da Query".
- **A estrutura:** aplicaremos o particionamento em tabelas históricas e mediremos a redução no tempo de resposta para filtros de data.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

O OTIMIZADOR DE QUERIES (QUERY OPTIMIZER)

Antes de executar qualquer SQL, o MySQL passa pelo Otimizador. Ele decide qual índice usar e qual o caminho mais barato (cost-based optimizer). Entender isso é entender que o SQL é **declarativo**: você diz o que quer, e o banco decide como buscar.

O MySQL possui um componente chamado **Cost-Based Optimizer (CBO)**. Quando você envia um comando SQL, o CBO não o executa imediatamente. Ele gera diversos "planos de execução" e escolhe aquele que ele acredita ter o menor custo computacional.

Exemplo: se você busca um cliente pelo ID, o otimizador sabe que deve ir direto para a Chave Primária. Se você busca pelo NOME, e não há índice, o otimizador é forçado a ler todas as páginas de dados do disco (Full Table Scan).

A Ferramenta de Ouro: o comando EXPLAIN. Ele é o raio-X da sua consulta. Ao colocar EXPLAIN antes do seu SELECT, o MySQL revela se ele usou índices, quantas linhas ele estimou ler e qual foi o tipo de junção aplicada.

Estatísticas de Tabela: o CBO utiliza amostragens de páginas de dados para estimar a cardinalidade (número de valores únicos). Se as estatísticas estiverem desatualizadas (ANALYZE TABLE), o otimizador pode escolher um plano subótimo.

Cálculo de Custo: o custo é uma métrica abstrata que soma o custo de I/O (leitura de disco/buffer pool) e o custo de CPU (comparação de chaves, ordenação).

Fórmula simplificada:

$$\text{Custo}_{\text{total}} = \left(\text{Páginas}_{\text{lidas}} \times \text{Custo}_{\text{I/O}}\right) + \left(\text{Linhas}_{\text{processadas}} \times \text{Custo}_{\text{CPU}}\right)$$

EXPLAIN ANALYZE (MySQL 8.0+): diferente do EXPLAIN tradicional, que mostra apenas a estimativa, o ANALYZE executa a query e compara a estimativa do otimizador com o tempo real de cada etapa, permitindo identificar gargalos de memória e CPU.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Índices: o atalho para o dado (O mapa do tesouro)

Um índice é uma estrutura de dados extra que o banco mantém para acelerar buscas. No motor **InnoDB**, a estrutura padrão é a **B-Tree (Árvore B)**, mas ele utiliza **B+Trees**, e não apenas B-Trees simples.

Obs.: **B+Tree vs. B-Tree:** Na B+Tree, todos os dados residem apenas nos nós folha. Os nós internos contêm apenas chaves de busca. Isso aumenta o fan-out (número de ponteiros por nó), reduzindo a altura da árvore e, consequentemente, o número de saltos de disco.

- **Clustered Index (Índice Agrupado):** no MySQL, a Chave Primária (PK) é o índice agrupado. Isso significa que os dados da tabela estão fisicamente ordenados no disco pela PK. É a busca mais rápida possível.
- **Secondary Indexes (Índices Secundários):** são índices criados em outras colunas (Ex.: e-mail). Eles funcionam como o índice de um livro técnico: você busca o termo "Cientista", ele te dá o número da página, e você vai até lá.
- **Índices compostos e a regra da esquerda:** se você criar um índice em (estado, cidade), ele funciona para buscas por estado ou por estado e cidade. Mas ele **não** funciona se você buscar apenas por cidade. É o que chamamos de Leftmost Prefix Rule.

Diferenças Principais (B-Tree vs. B+Tree):

Localização dos dados: em B-Trees, os dados podem estar em qualquer nó (interno ou folha). Em B+Trees, apenas os nós folhas contêm os registros de dados reais; nós internos contêm apenas chaves de navegação.

Estrutura de folha: em B+Trees, as folhas são conectadas por ponteiros, facilitando buscas sequenciais/intervalos.

Fan-out: como nós internos de B+Trees não armazenam dados, eles contêm mais chaves, diminuindo a altura da árvore e acelerando a busca.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Quando o InnoDB utiliza B+Trees:

- **Índices Primários e Secundários:** o InnoDB usa B+Trees para todos os seus índices.
- **Buscas por Intervalo:** o uso de B+Trees é ideal para SELECT * FROM table WHERE id > 100 AND id < 500, já que a árvore permite ir até a folha e percorrer as folhas sequencialmente.
- **Leitura/Escrita de Disco:** B+Trees minimizam as E/S de disco (I/O) necessárias, o que é crítico para performance de armazenamento em disco, tornando-as ideais para bancos relacionais como MySQL/InnoDB.

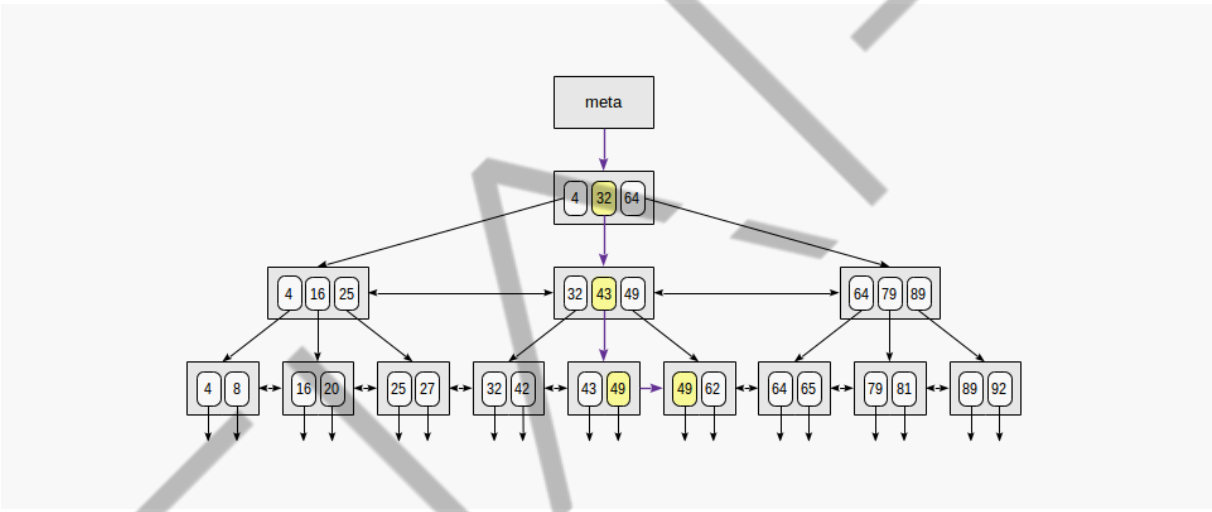


Figura 1 – Bancos relacionais
Fonte: postgrespro (2019)

PARTICIONAMENTO DE TABELAS: "DIVIDIR PARA CONQUISTAR"

Quando uma tabela chega na casa das centenas de milhões de linhas ou centenas de gigabytes, o índice sozinho começa a sofrer. O **Particionamento** divide a tabela em pedaços menores (partições) de forma transparente para a aplicação. Até os índices podem ficar lentos devido ao tamanho da árvore B. A divisão da tabela é física, mas a mantém logicamente como uma só. O particionamento visa mitigar a degradação de performance causada pelo aumento da profundidade da árvore de índices.

- **RANGE Partitioning:** o uso mais comum para Ciência de Dados. Imagine dividir os pedidos por ano: **Partição_2023**, **Partição_2024**.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

- **Partition Pruning:** se você faz um SELECT filtrando data_pedido em 2024, o MySQL "poda" (ignora) todas as outras partições e foca apenas na de 2024. Isso reduz drasticamente o I/O de disco. O motor de execução analisa as cláusulas WHERE durante a fase de análise sintática. Se o predicado de filtragem coincide com a chave de partição, o otimizador remove as partições irrelevantes do plano de execução antes mesmo de abrir os arquivos físicos.
- **Limitações técnicas:** em tabelas particionadas, todas as Chaves Únicas (incluindo a PK) devem obrigatoriamente incluir a coluna de partição. Isso ocorre porque o banco precisaria verificar cada partição individualmente para garantir a unicidade, o que anularia o ganho de performance.

Boas práticas e "Sargability": escrevendo queries "Buscáveis"

Uma query é Sargable (Search ARGumentABLE) quando ela permite que o banco (motor) use índices. A "Sargability" é a capacidade de um predicado ser convertido em um operador de busca de intervalo (range scan) em vez de um escaneamento completo (full scan).

Exemplo Não-Sargable (Lento): SELECT * FROM clientes WHERE YEAR(data_cadastro) = 2023;

Por que é ruim? O MySQL precisa aplicar a função YEAR em cada linha da tabela antes de comparar. Isso invalida o índice.

Exemplo Sargable (Rápido): SELECT * FROM clientes WHERE data_cadastro >= '2023-01-01' AND data_cadastro <= '2023-12-31';

Por que é bom? O valor é comparado diretamente com o que está gravado no índice, permitindo uma busca binária ultraveloz.

O problema das funções de manipulação: do ponto de vista algébrico, quando aplicamos uma função em uma coluna, como $\mathbf{f}(\mathbf{x})\ \mathbf{y}$, o índice (que está ordenado por $(\mathbf{x})$ torna-se inútil para localizar $\mathbf{y}$.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Dica avançada (Índices funcionais): no MySQL 8.0, é possível criar índices sobre expressões. Exemplo: CREATE INDEX idx_ano ON clientes ((YEAR(data_cadastro)));. Isso torna uma query anteriormente "non-sargable" em "sargable", pois o resultado da função já está pré-calculado e indexado.

Tipo de Acesso	Complexidade Algorítmica	Descrição Técnica
Const/Eq_ref	$O(1)$ ou $O(\log n)$	Acesso via Chave Primária ou Unique Key.
Ref	$O(\log n)$	Uso de índice não-único (pode retornar múltiplas linhas).
Range	$O(\log n + k)$	Busca de intervalo (ex: BETWEEN, >).
Index	$O(n)$	Full Index Scan: Percorre a árvore de índice inteira (mais rápido que a tabela).
ALL	$O(n)$	Full Table Scan: Leitura física completa de todos os blocos de dados.

Tabela 1: Tabela comparativa: estratégias de acesso
Fonte: Elaborado pelo autor (2026)

$O(1)$ - Complexidade constante

Este é o "Santo Graal" da performance. Significa que, não importa se sua tabela tem 100 linhas ou 1 bilhão de linhas, o tempo de resposta será **praticamente o mesmo**.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

No MySQL: ocorre no acesso const. Quando você busca por uma Chave Primária, o banco sabe exatamente em qual endereço físico o dado está. Ele não "procura"; ele "vai".

$O(\log n)$ - Complexidade logarítmica

É o comportamento padrão de uma **B+Tree** bem projetada. O logaritmo é o inverso da exponenciação. Na prática, se você dobrar o tamanho da sua tabela, o trabalho do banco de dados aumenta apenas **um passo**.

Por que é eficiente? Em uma árvore de índice, para encontrar um registro em 1 milhão de linhas, o MySQL faz aproximadamente 20 "pulos" entre os nós. Para 1 bilhão de linhas, ele faz apenas 30. O crescimento do esforço é muito lento em relação ao crescimento dos dados.

No MySQL: é o acesso do tipo ref ou eq_ref.

$O(\log n + k)$ - Busca de Intervalo (Range)

Aqui, o banco usa o índice para encontrar o início do intervalo $O(\log n + k)$ e depois percorre sequencialmente os próximos registros (k) que satisfazem a condição.

No MySQL: ocorre em WHERE preco BETWEEN 10 AND 50. Se o intervalo (k) for muito grande, a performance cai.

$O(n)$ - Complexidade Linear

Aqui mora o perigo. O trabalho é **diretamente proporcional** ao número de linhas. Se a tabela dobra, o tempo dobra. Se a tabela tem 1 bilhão de linhas, o banco lerá 1 bilhão de registros.

Index Scan: lê o índice inteiro (geralmente porque o índice é menor que a tabela, mas ainda assim lê tudo).

All (Full Table Scan): o pior cenário. O MySQL lê a tabela inteira do disco.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Resumo Visual da Eficiência

Para facilitar a explicação, imagine o esforço necessário para achar um nome em uma lista:

Notação	Estratégia	Esforço (Ex: 1.000 itens)	Esforço (Ex: 1.000.000 itens)
$O(1)$	Você já tem o número da página	1 operação	1 operação
$O(\log n)$	Busca binária (índice)	~10 operações	~20 operações
$O(n)$	Ler a lista do início ao fim	1.000 operações	1.000.000 operações

Tabela 2: Tabela de eficiência
Fonte: Elaborado pelo autor (2026)

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

Case: Otimização em Fintechs

Em empresas de meios de pagamento, o volume de transações é colossal. Uma query não otimizada para gerar o extrato de um cliente pode travar o banco. O uso de **Índices de Cobertura** (onde todas as colunas pedidas no SELECT já estão no índice) é a técnica que permite que o extrato apareça instantaneamente no app.

Case: E-commerce e o "Black Friday Protocol"

Em grandes varejistas, o volume de dados durante eventos promocionais aumenta em até 20x. O desafio não é apenas ler dados, mas evitar o **Lock de Tabelas** (quando uma escrita trava a leitura).

- **A Solução:** uso de **Read Replicas** (Réplicas de Leitura). Enquanto o banco principal recebe as vendas, o Cientista de Dados direciona suas queries de BI para uma réplica.
- **Técnica de Performance:** utilização de NOLOCK (ou READ UNCOMMITTED) em situações críticas para garantir que a consulta não espere por transações financeiras pesadas, priorizando a velocidade da informação em tempo real.

Case: Logística e Particionamento Geográfico

Uma empresa global de logística precisava analisar o tempo de entrega em 50 países. Uma única tabela de "Rastreamento" tornava as queries impraticáveis.

- **A Solução:** implementação de **Particionamento por LIST**. Em vez de datas, os dados foram divididos fisicamente por regiões geográficas (Ex: Partição_Latam, Partição_Europe).
- **Resultado:** consultas sobre a performance de entrega no Brasil passaram a ignorar 95% do restante da base de dados (Partition Pruning), reduzindo o tempo de processamento de minutos para segundos.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Tendência: Índices Invisíveis e IA

O MySQL 8 introduziu os **Invisible Indexes**. Eles permitem que o DBA marque um índice como invisível para o otimizador. Se a performance do sistema não cair após alguns dias, o índice pode ser deletado com segurança. Além disso, estamos vendo a ascensão de bancos de dados autônomos que sugerem a criação de índices baseados no comportamento de busca do Cientista de Dados.

Tendência: Vetores em SQL (AI-Ready Databases)

Com a explosão da Inteligência Artificial Generativa, os bancos de dados SQL tradicionais evoluíram para armazenar **Vector Embeddings**.

- **O que é:** agora, além de buscar por "Preço < 100", você pode fazer queries de similaridade semântica diretamente no SQL, como: "Encontre produtos visualmente parecidos com esta imagem".
- **Impacto:** o SQL deixa de ser apenas uma ferramenta de agregação numérica e passa a ser o motor que alimenta sistemas de recomendação baseados em LLMs (Large Language Models).

Tendência: Cloud Data Warehousing e o "Custo por Query"

Diferente dos servidores locais, na nuvem (BigQuery, Snowflake), você não paga pelo hardware, mas pela **quantidade de dados lidos**.

- **O Novo Perfil:** o especialista em SQL agora é também um gestor de custos. Uma query SELECT * em uma tabela de 1 Terabyte pode custar centenas de reais.
- **Skill do Futuro:** "Query FinOps" — a habilidade de otimizar o SQL não apenas para ser rápido, mas para ser financeiramente barato, utilizando colunas selecionadas e filtros altamente eficientes.

Dica de Ouro: O "Anti-Padrão" Oculto

Cuidado com o OR em tabelas gigantes: muitas vezes, o otimizador se confunde com a cláusula OR e desiste de usar índices. Uma tendência de alta performance é substituir o OR por UNION ALL, permitindo que o banco execute duas buscas indexadas separadas e junte o resultado no final.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

O EXPLAIN é seu melhor amigo: nunca tente otimizar no "achômetro". Olhe o plano de execução.

Índices têm custo: eles aceleram a leitura, mas deixam a escrita (INSERT, UPDATE) mais lenta, pois o banco precisa atualizar o mapa a cada mudança.

A tipagem importa (de novo): se você faz um JOIN entre uma coluna INT e uma VARCHAR, o MySQL converterá os tipos em tempo de execução e ignorará seus índices.

Sargability: evite funções em colunas dentro do WHERE. Deixe a coluna "limpa" para o índice trabalhar.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

GUILDER, S. Database Internals: A Deep Dive into How Distributed Data Systems Work. O'Reilly.

ORACLE CORPORATION. MySQL 8.4 Reference Manual - Optimization. Disponível em: dev.mysql.com.

SCHWARTZ, Baron. High Performance MySQL: Optimization, Backups, and Replication. 4th Edition. O'Reilly, 2022. (A "bíblia" da performance em MySQL).

WINAND, Markus. SQL Performance Explained. 2012. (Excelente foco em como os índices funcionam por dentro).

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

CTEs, SQL, MySQL, Agregações, Performance, OLAP, ETL.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com